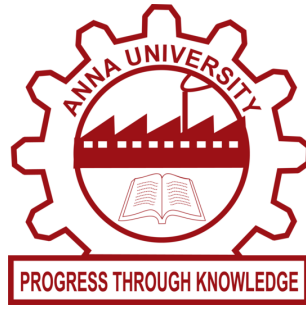




## **ANNA UNIVERSITY**

College of Engineering, Guindy

Department of Computer Science and Engineering  
(Data Science and Cyber Security)

### **PROJECT PROPOSAL (First Review)**

# **Phase-Based Container Security Framework: Adaptive Syscall Filtering and AI-Driven Threat Detection**

**Name** : Bharath D  
**Register Number** : 2025165027  
**Guide** : Dr. S. Bose

Submitted in partial fulfilment of the requirements for the award of the degree of

### **Master of Engineering**

Computer Science and Engineering  
(Data Science and Cyber Security)

Academic Year 2026–2027

# Contents

|                                                                |           |
|----------------------------------------------------------------|-----------|
| <b>Abstract</b>                                                | <b>3</b>  |
| <b>1 Introduction</b>                                          | <b>4</b>  |
| 1.1 Background . . . . .                                       | 4         |
| 1.2 The Syscall Interface as an Attack Surface . . . . .       | 5         |
| 1.3 Motivation for the Study . . . . .                         | 6         |
| 1.4 Objectives and Scope of Work . . . . .                     | 7         |
| <b>2 Summary of Related Work</b>                               | <b>9</b>  |
| 2.1 Automated Seccomp Policy Generation . . . . .              | 9         |
| 2.2 Machine-Learning-Based Syscall Anomaly Detection . . . . . | 10        |
| 2.3 Summary of Research Gaps . . . . .                         | 11        |
| <b>3 High-Level Block Diagram</b>                              | <b>13</b> |
| <b>4 Detailed Module Design</b>                                | <b>16</b> |
| 4.1 Phase I Modules . . . . .                                  | 16        |
| 4.1.1 Static Analysis Module . . . . .                         | 16        |
| 4.1.2 Dynamic Analysis Module . . . . .                        | 16        |
| 4.1.3 Feature Vector Extraction Module . . . . .               | 16        |
| 4.1.4 Adaptive Change-Point Detection Module . . . . .         | 17        |
| 4.1.5 Running Seccomp File Generation Module . . . . .         | 17        |
| 4.2 Phase II Modules . . . . .                                 | 17        |
| 4.2.1 eBPF-Based Syscall Capture Module . . . . .              | 17        |
| 4.2.2 Argument-Aware Feature Extraction Module . . . . .       | 18        |
| 4.2.3 Hybrid Anomaly Detection Module . . . . .                | 18        |
| 4.2.4 Risk Scoring and Response Module . . . . .               | 18        |
| <b>5 Implementation Details</b>                                | <b>19</b> |
| 5.1 Direct Syscall Resolution . . . . .                        | 19        |
| 5.2 Indirect Syscall Resolution . . . . .                      | 20        |
| 5.3 Sample Output . . . . .                                    | 20        |
| 5.4 Generated Static Seccomp File — Nginx . . . . .            | 21        |
| 5.5 Results and Benchmarking . . . . .                         | 21        |
| <b>6 Metrics for Evaluation</b>                                | <b>24</b> |
| 6.1 Metrics — Phase I . . . . .                                | 24        |
| 6.2 Metrics — Phase II . . . . .                               | 25        |
| 6.3 Datasets for Phase II Evaluation . . . . .                 | 25        |

|                                     |           |
|-------------------------------------|-----------|
| <b>7 Deliverables</b>               | <b>26</b> |
| 7.1 Phase I Deliverables . . . . .  | 26        |
| 7.2 Phase II Deliverables . . . . . | 26        |
| <b>8 Project Timeline</b>           | <b>27</b> |
| <b>9 References</b>                 | <b>28</b> |
| <b>Appendix: Base Paper</b>         | <b>29</b> |

# Abstract

Containerized deployments share the host operating system kernel across workloads, making the system call (syscall) interface the primary security boundary between an application and the host. While hundreds of syscalls are available to a typical container, only a small subset is genuinely required, and the remaining exposed syscalls needlessly widen the attack surface available to an adversary following a compromise. Existing phase-aware seccomp profiling approaches reduce this exposure by combining static binary analysis with dynamic eBPF-based monitoring, and by separating the container lifecycle into an initialization phase and a steady-state runtime phase. However, these approaches identify the transition between phases using a globally fixed similarity threshold and a syscall-frequency-only feature representation, both of which generalize poorly across workloads with different behavioral characteristics. Furthermore, even a minimized allowlist cannot prevent an attacker from misusing a legitimately permitted syscall through malicious arguments, since existing anomaly detection frameworks typically model only syscall identifiers and their sequential order, ignoring argument-level context.

This project proposes a two-phase container security framework that addresses both limitations. **Phase I** augments the static-and-dynamic seccomp profiling pipeline with an adaptive, sequence-aware running-phase segmentation mechanism: syscall frequency features are combined with bigram transition features to capture execution order, and a statistical change-point detector (Page-Hinkley Test / Bayesian Online Change-Point Detection) replaces the fixed similarity threshold, automatically calibrating the phase-transition point to each workload’s own behavior. **Phase II** extends eBPF-based runtime anomaly detection with argument-aware feature representations, feeding a hybrid Variational Autoencoder (VAE) and Isolation Forest ensemble that classifies live syscall sequences as normal or anomalous and triggers risk-based logging, alerting, or blocking. Together, the two phases provide defense in depth: proactive attack-surface minimization through an adaptively segmented seccomp policy, and continuous behavioral monitoring of the syscalls that remain permitted. A working static-analysis pipeline (direct and indirect syscall resolution via ELF disassembly and libc call-graph traversal) has been implemented and validated against representative container images (Nginx, Redis, MySQL), with results benchmarked against a peer-reviewed baseline.

# 1 Introduction

## 1.1 Background

Containerization has become the preferred deployment model for cloud-native applications due to its portability, scalability, and efficient resource utilization compared to traditional virtual machines. Unlike virtual machines, containers package applications and their dependencies while sharing the host operating system kernel, eliminating the need to boot separate guest operating systems for every workload. This shared-kernel architecture significantly improves deployment density and startup latency, but it introduces an important security challenge: since container isolation relies primarily on software mechanisms such as Linux namespaces and control groups (cgroups) rather than hardware-enforced virtualization boundaries, a compromised container may attempt to exploit kernel vulnerabilities to escalate privileges, escape the container boundary, or interfere with other workloads co-located on the same host.

The primary interface through which a containerized application interacts with the host kernel is the Linux system call (syscall) interface. Every operation performed by an application—including file access, process creation, memory management, network communication, and inter-process communication—is ultimately translated into one or more syscalls. Although the Linux kernel exposes several hundred syscalls (357 in Docker’s default Seccomp profile, for instance), most containerized applications require only a small, workload-specific subset of these during normal execution. By default, however, container processes often retain access to the full breadth of unnecessary syscalls, increasing the attack surface available to an adversary. Syscalls such as `ptrace`, `mount`, and `unshare` have been widely associated with privilege escalation and container escape attacks, precisely because they are rarely needed by ordinary application logic but are nonetheless permitted by default.

To mitigate this risk, Linux provides **Secure Computing Mode (seccomp)**, a kernel facility that restricts a process to a predefined allowlist of permitted syscalls. Through **seccomp-BPF**, these filtering rules are compiled into a Berkeley Packet Filter program and enforced efficiently within the kernel at syscall entry, with negligible runtime overhead. The central engineering challenge, however, is not the enforcement mechanism itself but the generation of an accurate and minimal allowlist: a policy that permits every syscall a given application genuinely needs across all of its legitimate code paths, while excluding everything else. Authoring such a policy by hand requires deep, simultaneous familiarity with both the target application’s internal behavior and the semantics of the syscall interface, which is impractical at the scale and diversity of modern container deployments.

While seccomp governs *what* a process is permitted to do, it says nothing about *how* a permitted syscall is used. **Extended Berkeley Packet Filter (eBPF)** complements this by enabling efficient runtime observability: eBPF allows small, kernel-verified programs to execute safely in response to events such as syscall entry and exit, without requiring custom

kernel modules or a kernel rebuild. This capability enables low-overhead collection of syscall metadata—identifiers, arguments, timestamps, and process context—at a fraction of the overhead of traditional user-space tracing tools such as `strace`, which rely on the significantly more expensive `ptrace` mechanism and incur a context switch for every intercepted syscall. Consequently, eBPF has become a foundational technology not only for automated seccomp profile generation (observing which syscalls an application actually exercises) but also for behavioral anomaly detection (observing *how* those syscalls are invoked over time).

This project leverages the complementary strengths of seccomp and eBPF through a two-phase framework. **Phase I** focuses on generating accurate, lifecycle-aware seccomp policies through improved syscall behavior modeling that distinguishes container initialization from steady-state execution. **Phase II** extends runtime protection through eBPF-based behavioral anomaly detection enriched with syscall argument information, so that malicious use of an already-permitted syscall can still be detected even when the syscall itself would otherwise pass a purely identifier-based allowlist check. Together, these two phases target both halves of the container security problem: reducing the syscalls an attacker can reach in the first place, and continuously watching how the remaining, legitimately required syscalls are actually used.

## 1.2 The Syscall Interface as an Attack Surface

A useful way to frame the problem this project addresses is to separate two distinct classes of syscall-related risk. The first is *exposure risk*: a syscall that an application never legitimately calls, but which remains permitted because the deployed policy is generic rather than workload-specific. Exposure risk is addressed by allowlist minimization—the smaller and more precisely tailored the permitted syscall set, the fewer avenues an attacker has even after gaining code execution inside the container. The second is *misuse risk*: a syscall that the application genuinely and legitimately calls, but which an attacker invokes with malicious arguments or in an anomalous sequence to achieve an unintended effect. A canonical example is `open()`: a web server legitimately calls `open()` routinely to serve static files, so no static allowlist can exclude it, yet the very same syscall targeting `/etc/shadow` or a container’s cgroup control files represents a fundamentally different, dangerous operation that identifier-only filtering cannot distinguish. Misuse risk can only be addressed by monitoring *how* a syscall is used at runtime, including its arguments, not merely whether it is called at all.

Existing research addresses these two risk classes largely in isolation: syscall-filtering literature focuses on allowlist minimization (exposure risk) using static and dynamic profiling, while syscall-based anomaly detection literature focuses on behavioral modeling (misuse risk) using sequence-based machine learning. The framework proposed in this project explicitly targets both, using a phased design in which Phase I minimizes exposure risk through an improved allowlist-generation pipeline, and Phase II reduces misuse risk through argument-aware behavioral monitoring layered on top of the Phase I output.

### 1.3 Motivation for the Study

Recent research has demonstrated the effectiveness of both automated seccomp policy generation and machine-learning-based syscall anomaly detection for enhancing container security. Hybrid profiling approaches that combine static binary analysis with dynamic runtime observation have shown that workload-specific seccomp allowlists can be generated more accurately than using either technique alone: static analysis provides broad code-path coverage without requiring the application to actually be exercised, while dynamic analysis captures behavior that is difficult or impossible to infer purely from a binary, such as syscalls reached only through data-dependent control flow. Furthermore, separating container execution into an initialization phase and a steady-state phase enables the construction of tighter security policies by removing permissions—such as `mount`, `pivot_root`, or `fork`—that are required only during container startup and represent unnecessary residual risk if left permitted for the container’s entire operational lifetime. In parallel, machine-learning-based anomaly detection frameworks, particularly those combining Variational Autoencoders (VAE) and Isolation Forests, have demonstrated the ability to identify abnormal syscall behavior in real time with high detection accuracy and low runtime overhead, without requiring hand-authored detection rules for every possible attack pattern.

Despite these advances, two important limitations remain, and both motivate the specific contributions of this project.

The first concerns phase-aware seccomp profile generation. Existing approaches typically identify the transition between initialization and steady-state execution using a globally fixed similarity threshold (for example, a cosine-similarity threshold of 0.8 applied uniformly across all workloads) evaluated over syscall-frequency-based features. Such methods do not account for workload-specific behavioral variation: a database server’s initialization sequence looks nothing like a web server’s, yet both are segmented using the same fixed constant. This makes segmentation prone to systematic misjudgment—either detecting the transition too early, while startup-only syscalls are still present, or too late, unnecessarily prolonging exposure to the broader initialization-phase allowlist. Compounding this, frequency-based feature representations are inherently order-blind: two time windows containing an identical multiset of syscalls but a completely different execution order are indistinguishable to a frequency-only feature vector, even though the underlying behavior—and therefore its security significance—may differ substantially.

The second limitation concerns runtime anomaly detection. Current syscall-based detection frameworks primarily analyze syscall identifiers and their sequential relationships while ignoring the arguments associated with each syscall invocation. This creates a significant blind spot, since many real-world container attacks exploit syscalls that are entirely legitimate from a sequence-modeling perspective, differing from benign use only in their arguments (an unexpected file path, an unusual socket address family, an unauthorized capability request). Notably,

one of the most directly relevant recent frameworks in this space explicitly identifies the absence of argument-level analysis as a known limitation of its own design, underscoring that this is an open and acknowledged gap in the field rather than a hypothetical concern.

This study is motivated by the observation that both limitations can be addressed while preserving the strengths—and the overall lifecycle-based design—of existing approaches. An adaptive, statistically-calibrated phase-transition mechanism combined with order-aware syscall features can improve the accuracy of phase-aware seccomp profiling without redesigning its underlying static-and-dynamic profiling workflow. Similarly, incorporating syscall argument information into the anomaly detection pipeline can enhance the ability to identify malicious behavior that sequence analysis alone cannot detect, without discarding the demonstrated effectiveness of VAE- and Isolation-Forest-based sequence modeling. By integrating these improvements, the proposed framework provides a defense-in-depth approach in which an adaptively segmented seccomp policy first reduces the syscalls available to an attacker, and continuous, argument-aware behavioral monitoring then detects misuse of whatever syscalls remain permitted.

## 1.4 Objectives and Scope of Work

The primary objective of this project is to design and implement a two-phase container security framework that proactively minimizes the syscall attack surface while continuously monitoring the runtime behavior of permitted syscalls to detect malicious activity.

The specific objectives are:

- To develop a hybrid static-and-dynamic seccomp profiling pipeline that combines binary disassembly, call graph analysis, and eBPF-based runtime monitoring to generate accurate and minimal workload-specific seccomp policies.
- To design an adaptive, sequence-aware segmentation mechanism that integrates syscall frequency, bigram transition features, and statistical change-point detection to accurately identify the transition from container initialization to steady-state execution.
- To develop an argument-aware hybrid anomaly detection model by combining a Variational Autoencoder (VAE) and an Isolation Forest (iForest) for real-time detection of abnormal syscall behavior captured through eBPF.
- To implement a risk-scoring and response mechanism that classifies detected anomalies based on severity and initiates appropriate actions, including logging, alert generation, and runtime blocking.
- To evaluate the proposed framework by comparing it with existing baseline approaches in terms of phase segmentation accuracy, seccomp policy quality, anomaly detection performance, and runtime overhead.

This work focuses on the design, implementation, and evaluation of the proposed profiling, segmentation, and anomaly detection mechanisms using representative containerized applications such as **Nginx**, **Redis**, and **MySQL**. The implementation covers static analysis, dynamic syscall monitoring, adaptive phase detection, seccomp policy generation, argument-aware anomaly detection, and risk-based response mechanisms.

The scope excludes deployment in large-scale production environments and integration with container orchestration platforms such as Kubernetes. Automated CI/CD integration, multi-node orchestration, and distributed policy management are considered future extensions beyond the scope of this work.

## 2 Summary of Related Work

Existing research relevant to this project can be broadly classified into two areas: **automated seccomp policy generation for container attack surface reduction**, which is the primary foundation for Phase I, and **machine-learning-based syscall anomaly detection for runtime security**, which is the primary foundation for Phase II. This section summarizes the methodologies and limitations of the most closely related works in each area.

### 2.1 Automated Seccomp Policy Generation

The base work most directly underlying Phase I of this project is Chen *et al.*'s **Phase-based System Call Filtering (PSF)** framework [1], which combines static binary analysis (via the Confine call-graph tool) with dynamic eBPF-based tracing to construct a Static Seccomp File (S-SF) and Dynamic Seccomp File (D-SF), merged into an Initialize Seccomp File (I-SF) for the container's startup phase. PSF further separates container execution into initialization and steady-state phases using a fixed-size time-window segmentation of the live syscall stream, computing cosine similarity between consecutive windows' frequency-and-type feature vectors and declaring the phase transition once similarity stabilizes above a globally fixed threshold of 0.8 over a 5-second sliding window. The resulting Running Seccomp File (R-SF) is shown, across ten popular Docker Hub images (including Redis, MySQL, Nginx, httpd, Python, Java, Golang, Ubuntu, CentOS, and Fedora), to substantially reduce the syscall count relative to both the initialization-phase whitelist and Docker's 357-syscall default—for example, Redis's syscall set is reduced from 134 (S-SF) to 69 (R-SF). PSF is validated against ChestNut's dynamic analysis results and shown to achieve comparable accuracy while requiring substantially less manual configuration than prior static-only approaches such as Temporal Specialization (TS). This framework serves as the primary foundation and benchmark baseline for the Phase I contribution of this project; its central limitations—a fixed, non-adaptive similarity threshold, and syscall-frequency-only features that cannot capture execution order—are precisely what Phase I is designed to address.

Kang *et al.*'s **BeaCon** framework [2] generates automatic container security policies using environment-aware dynamic analysis: an eBPF-based monitor observes syscalls and capability usage while an emulated-environment exploration component systematically probes execution paths that would not otherwise be exercised by a single representative run, combined with a scoring mechanism to prioritize which paths to explore. This addresses one of the central weaknesses of purely observational dynamic analysis—coverage of rarely-triggered code paths—and demonstrates low runtime overhead while mitigating realistic kernel-level attacks in evaluation. Its principal limitation is that effectiveness is fundamentally bounded by the quality and completeness of the emulated exploration; paths that the emulation does not consider remain unobserved, and the approach may still miss genuinely rare execution paths that

only manifest under specific, unmodeled runtime conditions.

Older static-only and hybrid approaches referenced within the PSF study’s own related-work discussion further contextualize the design space: Confine [1] constructs a call graph combining direct and libc-mediated calls for improved static accuracy but may include redundant calls not actually reachable at runtime; Sysfilter, using binary-only analysis via Nibbler, removes more unnecessary calls than Confine at the cost of requiring binary-only heuristics; Abhaya generates policies directly from source code, which is unavailable for many deployed binary-only container images; and Ghavamnia *et al.*’s Temporal Specialization divides syscall needs into initialization and runtime phases by analyzing the call graph, but—similarly to PSF—relies on manually defined phase boundaries and is limited to a small set of common containers. Dynamic-only tools such as `strace`, Sysdig, and eBPF-based tracers collect syscall traces during actual execution, compensating for the blind spots of static analysis but requiring a real, resource-consuming execution environment and potentially still missing rare or complex execution paths that are not exercised during profiling.

## 2.2 Machine-Learning-Based Syscall Anomaly Detection

Zehra *et al.*’s **DeSFAM** framework [3] is the primary foundation for the Phase II contribution of this project. DeSFAM combines hybrid static-dynamic profiling with a **SyscallAD** detection component built on an ensemble of a Variational Autoencoder and an Isolation Forest, further incorporating MITRE ATT&CK- and CVE-informed risk scoring, with policy enforcement carried out through combined eBPF and Linux Security Module (LSM) hooks. Reported evaluation results include 94% precision, 90% recall, sub-millisecond detection latency, and under 1% runtime overhead, demonstrating that high-accuracy behavioral anomaly detection is achievable with negligible cost to the monitored workload. Critically, the authors explicitly self-identify the absence of argument-level syscall analysis as a limitation of their own approach—their model evaluates syscall identifiers and sequence information, but not the arguments passed to each syscall—which is precisely the gap that Phase II of this project is designed to close.

Kim *et al.* [4] apply eBPF/Tetragon-based tracing combined with n-gram syscall features and conventional machine-learning classifiers specifically to detect cryptojacking containers, reporting classification accuracy up to 99.75%. While highly effective for the specific threat class studied, the authors note that n-gram features derived from known cryptomining behavior may not generalize well to novel or previously unseen mining techniques, illustrating a broader limitation of narrowly-trained anomaly models: strong performance on the threat classes represented in training data does not guarantee robustness against structurally different, unseen attack patterns.

Zehra *et al.*’s companion work, **FedMon** [7], extends the DeSFAM detection architecture to a federated setting, deploying per-cluster eBPF agents that share only VAE and Isolation Forest

model updates rather than raw syscall data, reporting 94% precision, 91% recall, and a 60% reduction in inter-cluster bandwidth relative to centralized data sharing. This is relevant primarily as a scalability extension rather than a detection-accuracy improvement; its main identified limitation is that cross-cluster heterogeneity in workload behavior can affect the convergence of the shared model, a concern outside the single-node scope of this project.

Complementary eBPF-based detection work targets narrower attack surfaces: Zhou *et al.* [5] propose lightweight eBPF-based file-access monitoring with multimodal path-semantic features for real-time, fine-grained detection of abnormal file access as an attack precursor, though the approach’s scope is explicitly limited to file-access-based indicators and does not address the broader syscall behavior surface. Ryu *et al.* [6] combine eBPF-captured flow and syscall traces with multi-class machine-learning classification to achieve 98.39% host-based malicious-container detection accuracy, though the evaluation is conducted over a comparatively limited set of malware and benign scenarios. Taguchi *et al.* [8] implement an unsupervised, quantization-aware autoencoder performing fixed-point inference directly within the kernel via BPF maps, achieving roughly a  $70\times$  throughput improvement over user-space intrusion detection systems, though the achievable model size and complexity are constrained by eBPF’s kernel-imposed stack and instruction-count limits. Lin *et al.* [10] propose **eBPF-Guard**, a multi-level container-escape detection method combining syscall-level, call-feature-level, and log-level eBPF probes with an LLM-based analysis stage capable of surfacing previously unseen escape indicators, at the cost of additional inference latency and resource overhead introduced by the LLM stage. Finally, Her *et al.* [9] provide a comparative evaluation of several widely deployed eBPF-based security tools—KubeArmor, Falco, Tetragon, and Tracee—across hook-point coverage, policy scope, and runtime overhead, serving as a survey rather than a novel detection mechanism, but providing useful context for tool selection and instrumentation design decisions within this project.

## 2.3 Summary of Research Gaps

Table 1 summarizes the ten works reviewed above, alongside their core methodology and principal self-identified or externally observed limitation. Two research gaps emerge consistently across this body of work and directly motivate the two phases of this project. First, existing phase-aware seccomp profiling techniques rely on a globally fixed similarity threshold and frequency-based, order-insensitive syscall representations, limiting their ability to accurately identify lifecycle transitions across workloads with different behavioral characteristics—addressed in this project by Phase I’s adaptive, sequence-aware segmentation mechanism. Second, current hybrid anomaly detection frameworks, including the most closely related and highest-performing among them, operate without incorporating syscall argument information, preventing them from distinguishing malicious use of legitimate syscalls when the syscall sequence itself appears entirely normal—addressed in this project by Phase II’s argument-aware

feature representation.

Table 1: Summary of Reviewed Literature

| S.No | Publication                                                | Methodology                                                                                                     | Limitation                                                                                    |
|------|------------------------------------------------------------|-----------------------------------------------------------------------------------------------------------------|-----------------------------------------------------------------------------------------------|
| 1    | Chen <i>et al.</i> , PSF, IEEE TCC 2025 [1]                | Static + dynamic eBPF profiling (S-SF, D-SF, I-SF); cosine-similarity segmentation (0.8, 5s) for R-SF           | Fixed similarity threshold, not adaptive to workload; frequency-only features are order-blind |
| 2    | Zehra <i>et al.</i> , DeSFAM, IEEE Access 2025 [3]         | Hybrid static-dynamic profiling; SyscallAD (VAE + iForest); MITRE ATT&CK/CVE risk scoring; eBPF+LSM enforcement | No argument-level syscall analysis (self-identified)                                          |
| 3    | Kim <i>et al.</i> , Electronics 2025 [4]                   | eBPF/Tetragon tracing; n-gram features + ML classifiers                                                         | N-gram features may not generalize to novel/unseen mining techniques                          |
| 4    | Zehra <i>et al.</i> , FedMon, ICET 2025 [7]                | Per-cluster eBPF agents + federated learning; shared VAE + iForest model updates only                           | Cross-cluster heterogeneity can affect model convergence                                      |
| 5    | Her <i>et al.</i> , IEEE Access 2025 [9]                   | Comparative evaluation: KubeArmor, Falco, Tetragon, Tracee; hook-point, policy-scope, overhead analysis         | Comparative survey only — no new detection mechanism proposed                                 |
| 6    | Kang <i>et al.</i> , BeaCon, Computers & Security 2026 [2] | eBPF monitor for syscalls/capabilities; emulated environment exploration + scoring                              | Effectiveness depends on quality of emulated exploration; may miss rarely triggered paths     |
| 7    | Zhou <i>et al.</i> , Mathematics 2026 [5]                  | Lightweight eBPF file-access monitoring; multimodal path-semantic features                                      | Scope limited to file-access-based attack precursors only                                     |
| 8    | Ryu <i>et al.</i> , CMC 2026 [6]                           | eBPF flow + syscall traces; multi-class ML classification                                                       | Evaluated on a limited set of malware/benign scenarios                                        |
| 9    | Taguchi <i>et al.</i> , IEICE 2026 [8]                     | Unsupervised autoencoder, quantization-aware training; fixed-point in-kernel inference via BPF maps             | eBPF stack/instruction-count limits constrain achievable model size                           |
| 10   | Lin <i>et al.</i> , eBPF-Guard, EMSE 2026 [10]             | Multi-level eBPF probes (syscalls + call features + logs); LLM-based analysis stage                             | Runtime LLM inference adds latency/resource cost                                              |

### 3 High-Level Block Diagram

The proposed framework adopts a two-phase, defense-in-depth architecture that combines proactive attack surface reduction with continuous runtime monitoring. **Phase I** generates a workload-specific, lifecycle-aware seccomp policy by integrating static binary analysis, eBPF-based runtime profiling, and adaptive sequence-aware phase segmentation. The resulting seccomp allowlist minimizes the set of syscalls available to the container while preserving normal application functionality. Building upon this restricted execution environment, **Phase II** continuously monitors the permitted syscalls using eBPF-based runtime tracing; the captured syscall sequences, enriched with argument-level information, are analyzed using a hybrid anomaly detection model to identify deviations from normal behavior, with detected anomalies assigned a risk score and mitigated through logging, alerting, or blocking.

The high-level architecture of the proposed framework is illustrated in Figure 1 (Phase I) and Figure 2 (Phase II).

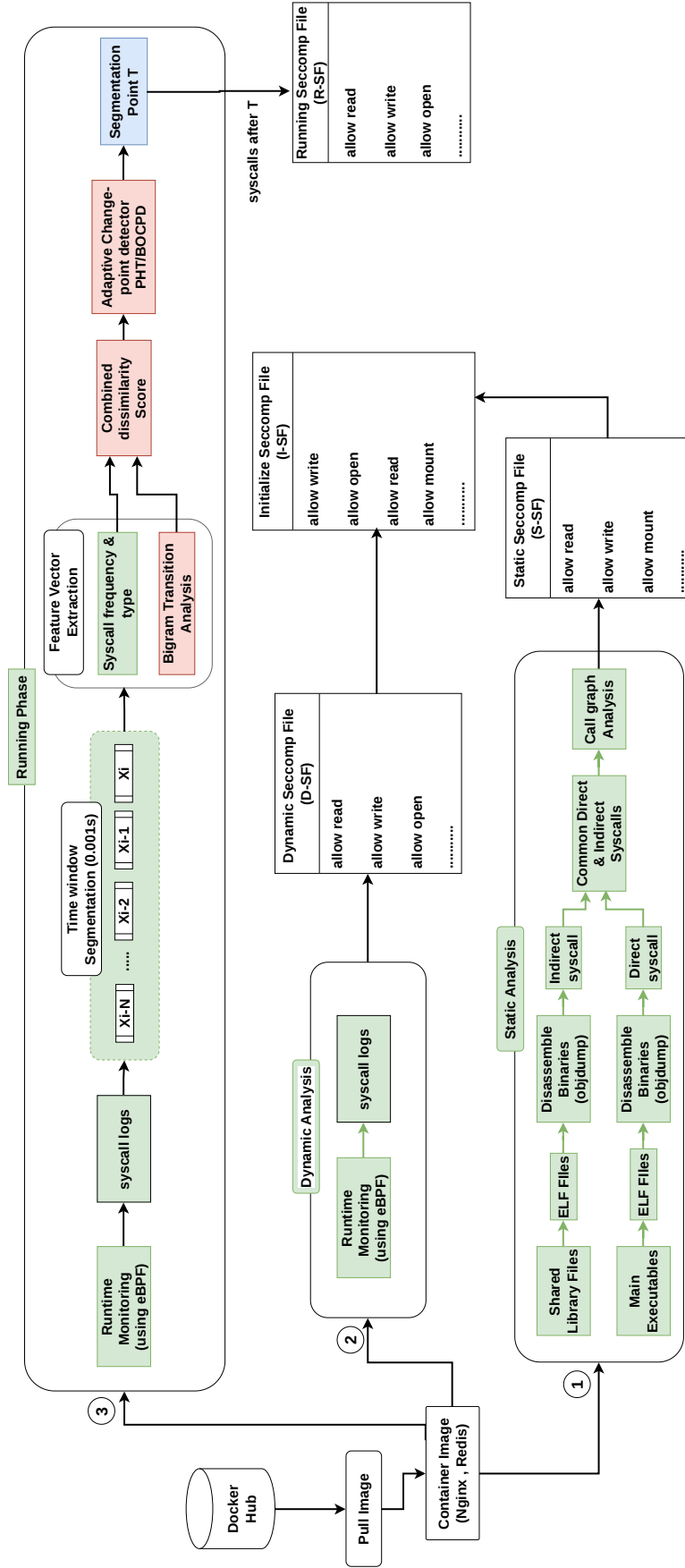


Figure 1: High-Level Architecture — Phase I (Seccomp Profile Generation)

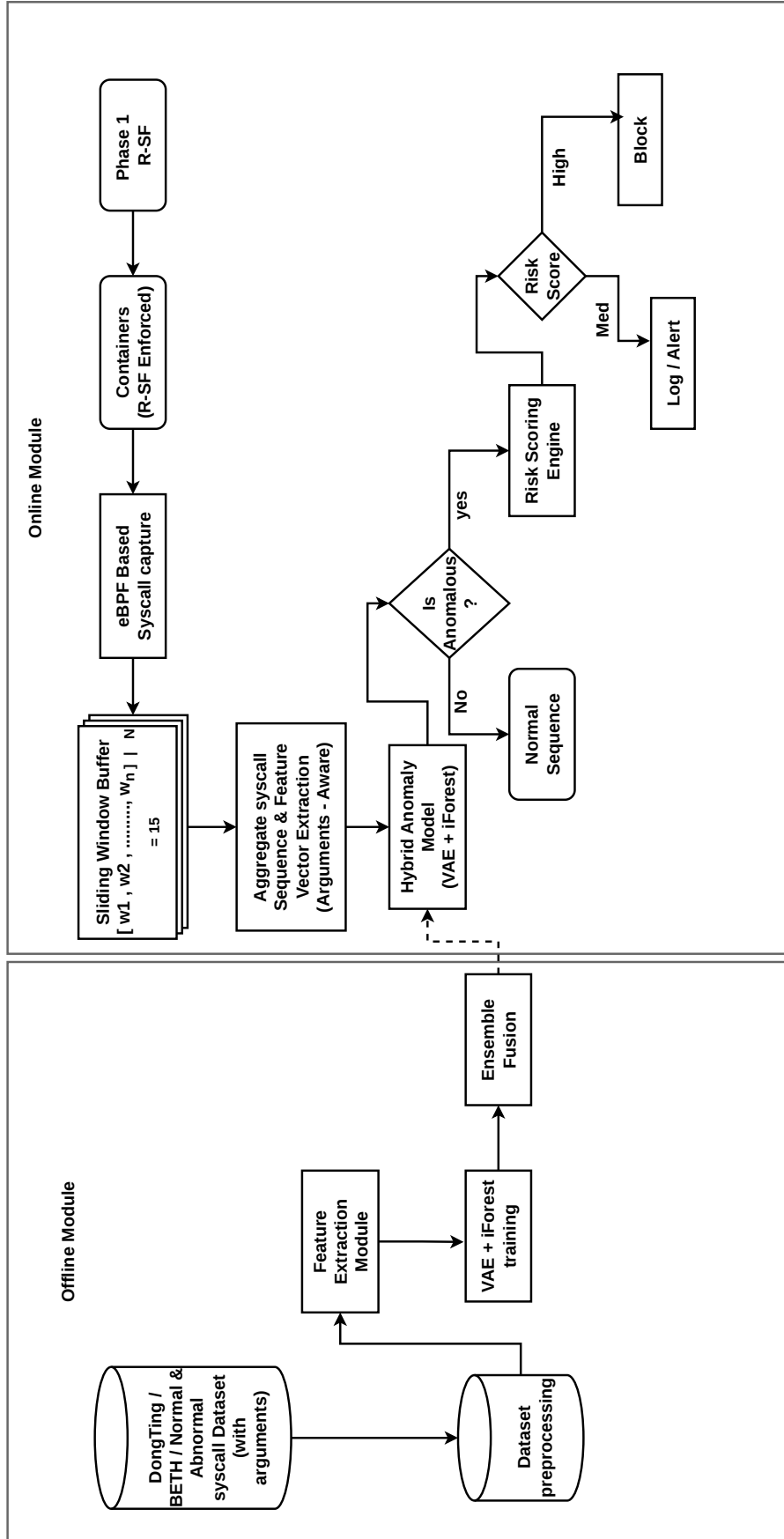


Figure 2: High-Level Architecture — Phase II (Real-Time Anomaly Detection)

## 4 Detailed Module Design

This section describes each module of the proposed framework along with its explicit input and output, for both Phase I (seccomp profiling) and Phase II (runtime anomaly detection).

### 4.1 Phase I Modules

#### 4.1.1 Static Analysis Module

Accepts a container image and generates the **Static Seccomp File (S-SF)**. The container image is scanned to extract the main executable and shared-library ELF binaries; each binary is disassembled (via `objdump/Capstone`), and both direct syscall instructions and library-mediated (indirect) syscalls are identified through call-graph traversal of the resolved libc call graph.

|               |                                                                                                                                                                                                      |
|---------------|------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <b>Input</b>  | Container image; extracted ELF binaries (main executable and shared libraries); pre-built glibc/musl call graph.                                                                                     |
| <b>Output</b> | Static Seccomp File (S-SF) — a de-duplicated list of syscalls statically reachable from the binary, each tagged with its resolution path (direct instruction address, or indirect call-graph trace). |

#### 4.1.2 Dynamic Analysis Module

Accepts a running container instance and generates the **Dynamic Seccomp File (D-SF)**. An eBPF program monitors syscall entry/exit events during representative workload execution, capturing syscall identifiers, arguments, timestamps, and process context. A crash-information feedback loop, seeded from the S-SF as a candidate allowlist, iteratively identifies and adds syscalls that were denied and caused a failure, converging on a whitelist under which the workload runs cleanly.

|               |                                                                                                                                                                |
|---------------|----------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <b>Input</b>  | Running container instance; candidate whitelist seeded from S-SF; representative workload/test traffic.                                                        |
| <b>Output</b> | Dynamic Seccomp File (D-SF) — syscalls observed and confirmed necessary during live execution. Merged with S-SF to produce the Initialize Seccomp File (I-SF). |

#### 4.1.3 Feature Vector Extraction Module

Processes the runtime syscall stream captured by eBPF during the running phase. The stream is divided into fixed-size time windows (e.g., 0.001s), and two complementary feature vectors are extracted per window: a syscall frequency-and-type vector (base-paper feature) and a bigram transition vector capturing syscall execution order (proposed addition).

|               |                                                                                                                                          |
|---------------|------------------------------------------------------------------------------------------------------------------------------------------|
| <b>Input</b>  | Live syscall stream from eBPF; time-window size parameter.                                                                               |
| <b>Output</b> | Per-window combined feature vector (frequency-and-type + bi-gram transition); pairwise dissimilarity scores between consecutive windows. |

#### 4.1.4 Adaptive Change-Point Detection Module

Identifies the transition from initialization to steady-state phase. Given the sequence of dissimilarity scores from the Feature Vector Extraction Module, applies a statistical change-point detection algorithm (Page-Hinkley Test or Bayesian Online Change-Point Detection), with the detection threshold calibrated adaptively to the workload’s own observed behavioral variance, rather than a fixed global constant.

|               |                                                                                                                                 |
|---------------|---------------------------------------------------------------------------------------------------------------------------------|
| <b>Input</b>  | Sequence of dissimilarity scores across time windows.                                                                           |
| <b>Output</b> | Segmentation point $T$ — the time window at which the container is determined to have transitioned into steady-state execution. |

#### 4.1.5 Running Seccomp File Generation Module

Generates the **Running Seccomp File (R-SF)** using the segmentation point  $T$ . eBPF monitoring continues beyond  $T$ , and syscalls observed during steady-state execution are collected into the final, tightened allowlist enforced for the remainder of the container’s execution.

|               |                                                                                                                                                 |
|---------------|-------------------------------------------------------------------------------------------------------------------------------------------------|
| <b>Input</b>  | Segmentation point $T$ ; live syscall stream beyond $T$ .                                                                                       |
| <b>Output</b> | Running Seccomp File (R-SF) — the minimized, steady-state syscall allowlist, enforced via seccomp-BPF for the container’s operational lifetime. |

## 4.2 Phase II Modules

### 4.2.1 eBPF-Based Syscall Capture Module

Continuously captures live syscalls from R-SF-restricted running containers into a fixed-depth sliding-window buffer for downstream feature extraction.

|               |                                                                                                     |
|---------------|-----------------------------------------------------------------------------------------------------|
| <b>Input</b>  | Live, R-SF-restricted container syscall activity.                                                   |
| <b>Output</b> | Sliding-window buffer of recent syscall events (identifier, arguments, timestamp, process context). |

### 4.2.2 Argument-Aware Feature Extraction Module

Aggregates syscall sequences per window into feature vectors that include argument-level context (e.g., target file paths, socket address families, capability flags), in addition to syscall identifiers and ordering.

|               |                                                                                           |
|---------------|-------------------------------------------------------------------------------------------|
| <b>Input</b>  | Sliding-window buffer of syscall events.                                                  |
| <b>Output</b> | Argument-aware feature vector per window, suitable for input to the hybrid anomaly model. |

### 4.2.3 Hybrid Anomaly Detection Module

A VAE + Isolation Forest ensemble, trained offline on the DongTing and BETH datasets, evaluates each window's feature vector and classifies it as Normal or Anomalous.

|               |                                                                                            |
|---------------|--------------------------------------------------------------------------------------------|
| <b>Input</b>  | Argument-aware feature vector (online); offline-trained VAE + iForest ensemble parameters. |
| <b>Output</b> | Per-window classification (Normal / Anomalous).                                            |

### 4.2.4 Risk Scoring and Response Module

Anomalous sequences are scored by a risk-scoring engine; medium-risk sequences trigger logging/alerting, and high-risk sequences trigger active blocking.

|               |                                                                                   |
|---------------|-----------------------------------------------------------------------------------|
| <b>Input</b>  | Anomalous window classification and its feature vector.                           |
| <b>Output</b> | Risk score (Medium/High) and corresponding response action (Log/Alert, or Block). |

## 5 Implementation Details

The current implementation covers the **Static Analysis Module** of Phase I in full, representing approximately 20% of the overall planned project implementation. This module performs both direct syscall resolution (disassembly-based) and indirect syscall resolution (libc call-graph-based) on real container binaries, and produces a working Static Seccomp File (S-SF) in OCI Seccomp JSON format.

### 5.1 Direct Syscall Resolution

The direct analyzer disassembles each executable section of a target binary using Capstone, classifies instructions that trigger a syscall (either a raw `syscall/int 0x80` instruction, or a call to a known libc syscall wrapper resolved via the PLT/GOT), and then performs a backward register-trace to recover the constant syscall number loaded prior to that instruction. Listing 3 shows the instruction-classification logic, and Listing 4 shows the backward-tracing logic, including explicit handling of caller-saved register clobbering across intervening function calls.

```
1 def _classify_syscall_instruction(self, instruction, architecture, ...):
2     mnemonic = instruction.mnemonic
3
4     # Raw machine syscall instruction
5     if mnemonic == "syscall":
6         return "raw_syscall"
7     if mnemonic == "int" and instruction.operands[0].imm == 0x80:
8         return "raw_int80"
9     if mnemonic == "sysenter":
10        return "raw_sysenter"
11
12    # Function calls to syscall wrapper functions
13    if mnemonic in {"call", "jmp"}:
14        callee_name = plt_symbols.get(target_addr) or func_symbols.get(target_addr)
15        if self._is_syscall_wrapper_name(callee_name):
16            return "wrapper_call"
```

Figure 3: Classifying syscall-triggering instructions

```
1 def _resolve_syscall_number(self, instructions, syscall_index, target_reg="rax", ...):
2     window = instructions[max(0, syscall_index - self.backtrack_window):syscall_index]
3     reg_state = {}
4
5     for insn in window:
6         if self._is_call_or_ret(insn):
7             # caller-saved registers are clobbered by any intervening call
8             for reg in ["rax", "rcx", "rdx", "rsi", "rdi", ...]:
9                 reg_state.pop(reg, None)
10            continue
11
12            if insn.mnemonic in {"mov", "movabs"} and src.type == CS_OP_IMM:
13                reg_state[dest_reg] = int(src.imm) # constant loaded
14
15    if canonical_target in reg_state:
16        return ResolutionResult(number=reg_state[canonical_target], resolution="static")
17    return ResolutionResult(number=None, resolution="unresolved")
```

Figure 4: Backward register trace to resolve the syscall number

## 5.2 Indirect Syscall Resolution

The indirect analyzer extracts a binary’s imported (undefined) symbols from its `.dynsym` section, normalizes each imported name into candidate libc call-graph node names (stripping ELF symbol-version suffixes, toggling underscore prefixes, and stripping numeric suffixes such as the “64” in `stat64`), and then performs an iterative, cycle-protected depth-first search over the resolved libc call graph until a `syscall(N)` leaf node is reached. Listing 5 shows the name normalization logic, and Listing 6 shows the graph traversal.

```
1 def _resolve_callgraph_function_name(function, analyzer):
2     candidates = [function]
3     if "@" in function:
4         candidates.append(function.split("@", 1)[0])           # strip version suffix
5     clean = candidates[-1]
6     if clean.startswith("__") and len(clean) > 2:
7         candidates.append(clean[2:])                           # __foo -> foo
8     elif clean.startswith("_") and len(clean) > 1:
9         candidates.append(clean[1:])                           # _foo -> foo
10    else:
11        candidates.append("__" + clean)                         # foo -> __foo
12    if clean.endswith("64"):
13        candidates.append(clean[:-2])                           # stat64 -> stat
14
15    for cand in candidates:
16        if analyzer.has_function(cand):
17            return cand
18    return None
```

Figure 5: Normalizing imported symbol names to call-graph node names

```
1 def get_syscalls_with_paths(self, function):
2     results = []
3     visited_nodes = set()
4     stack = [(function, [])]
5
6     while stack:
7         node, path = stack.pop()
8         if node in visited_nodes:
9             continue
10        visited_nodes.add(node)
11
12        syscall_num = self.parse_syscall_node(node)
13        if syscall_num is not None:
14            results.append(SyscallPath(number=syscall_num, path=path + [node]))
15            continue                                             # leaf reached, stop expanding
16
17        for child in self.graph.get(node, set()):
18            if child not in visited_nodes:
19                stack.append((child, path + [node]))
20
21    return results
```

Figure 6: DFS traversal of the libc call graph to a syscall leaf node

## 5.3 Sample Output

Figure 2 shows a representative excerpt of the resolved syscall report generated by the pipeline for a real container binary, including both direct resolutions (address and confidence) and indi-

rect resolutions (full call-graph trace path from the imported function down to the `syscall` (N) leaf node).

Table 2: Excerpt of Resolved Syscall Report

| Number | Name       | Resolution                                                                                           |
|--------|------------|------------------------------------------------------------------------------------------------------|
| 126    | capset     | Direct @ 0x5e345 (high confidence)                                                                   |
| 207    | io_destroy | Direct @ 0x5fb8b (high confidence)                                                                   |
| 321    | bpf        | Direct @ 0x61494 (high confidence)                                                                   |
| 186    | gettid     | Direct @ 0x6290f (high confidence)                                                                   |
| 330    | pkey_alloc | Indirect: pthread_cond_signal → syscall(330)                                                         |
| 38     | setitimer  | Indirect: pthread_cond_signal →<br>__pthread_cond_signal_2_0 → calloc →<br>__setitimer → syscall(38) |

## 5.4 Generated Static Seccomp File — Nginx

Table 3 shows the complete, unedited Static Seccomp File generated by the implemented pipeline for the Nginx binary, in OCI Seccomp JSON format — the actual artifact produced by the tool, containing all 124 resolved syscalls (the same count reported in Section 5.4 and benchmarked against the base paper in Figure 7).

## 5.5 Results and Benchmarking

The generated Static Seccomp File (S-SF) for each analyzed binary was benchmarked against the corresponding value reported in the PSF base paper [1] for the same application, and separately against Docker’s default Seccomp profile (357 allowed syscalls) to quantify attack-surface reduction. Figures 7 and 8 present these comparisons for three representative binaries analyzed to date (Nginx, Redis, MySQL).

Table 3: Seccomp system calls allowed by the security profile.

| No. | System Call     | No. | System Call     | No. | System Call            | No. | System Call     |
|-----|-----------------|-----|-----------------|-----|------------------------|-----|-----------------|
| 1   | accept          | 32  | ftruncate       | 63  | nanosleep              | 94  | sched_yield     |
| 2   | accept4         | 33  | futex           | 64  | newfstatat             | 95  | sendfile        |
| 3   | access          | 34  | get_kernel_syms | 65  | open                   | 96  | sendmsg         |
| 4   | arch_prctl      | 35  | getcwd          | 66  | openat                 | 97  | sendto          |
| 5   | bind            | 36  | getdents        | 67  | pkey_alloc             | 98  | set_robust_list |
| 6   | bpf             | 37  | geteuid         | 68  | pkey_free              | 99  | setgid          |
| 7   | capset          | 38  | getpid          | 69  | pkey_mprotect          | 100 | setgroups       |
| 8   | chdir           | 39  | getppid         | 70  | prctl                  | 101 | setitimer       |
| 9   | chmod           | 40  | getrandom       | 71  | pread64                | 102 | setpriority     |
| 10  | chown           | 41  | getrlimit       | 72  | preadv2                | 103 | setrlimit       |
| 11  | clock_getres    | 42  | getsockname     | 73  | pwrite64               | 104 | setsid          |
| 12  | clock_gettime   | 43  | getsockopt      | 74  | pwritev                | 105 | setsockopt      |
| 13  | clone           | 44  | gettid          | 75  | pwritev2               | 106 | setuid          |
| 14  | close           | 45  | gettimeofday    | 76  | query_module           | 107 | shutdown        |
| 15  | connect         | 46  | getuid          | 77  | read                   | 108 | socket          |
| 16  | copy_file_range | 47  | io_destroy      | 78  | readlink               | 109 | socketpair      |
| 17  | creat           | 48  | io_getevents    | 79  | readv                  | 110 | stat            |
| 18  | create_module   | 49  | io_setup        | 80  | recvfrom               | 111 | statfs          |
| 19  | dup             | 50  | ioctl           | 81  | recvmsg                | 112 | sysinfo         |
| 20  | dup2            | 51  | kill            | 82  | rename                 | 113 | tgkill          |
| 21  | epoll_create    | 52  | listen          | 83  | rmdir                  | 114 | time            |
| 22  | epoll_ctl       | 53  | lseek           | 84  | rt_sigaction           | 115 | umask           |
| 23  | epoll_wait      | 54  | lstat           | 85  | rt_sigprocmask         | 116 | umount2         |
| 24  | eventfd2        | 55  | madvise         | 86  | rt_sigreturn           | 117 | uname           |
| 25  | execve          | 56  | memfd_create    | 87  | rt_sigsuspend          | 118 | unlink          |
| 26  | execveat        | 57  | mkdir           | 88  | sched_get_priority_max | 119 | uselib          |
| 27  | exit            | 58  | mlock2          | 89  | sched_get_priority_min | 120 | utimes          |
| 28  | exit_group      | 59  | mmap            | 90  | sched_getparam         | 121 | vfork           |
| 29  | fadvise64       | 60  | mprotect        | 91  | sched_getscheduler     | 122 | wait4           |
| 30  | fcntl           | 61  | mremap          | 92  | sched_setaffinity      | 123 | write           |
| 31  | fstat           | 62  | munmap          | 93  | sched_setscheduler     | 124 | writev          |

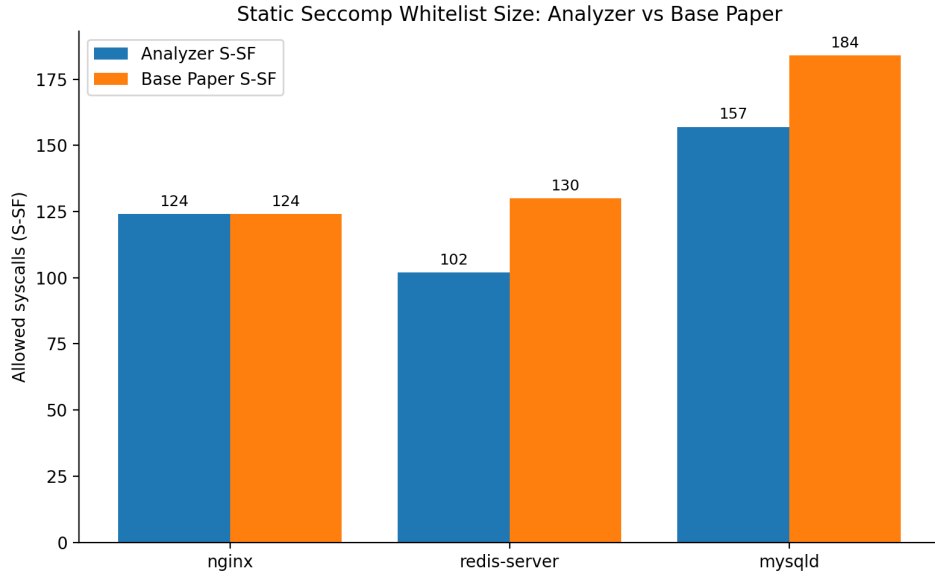


Figure 7: Static Seccomp Whitelist Size: Implemented Analyzer vs. Base Paper

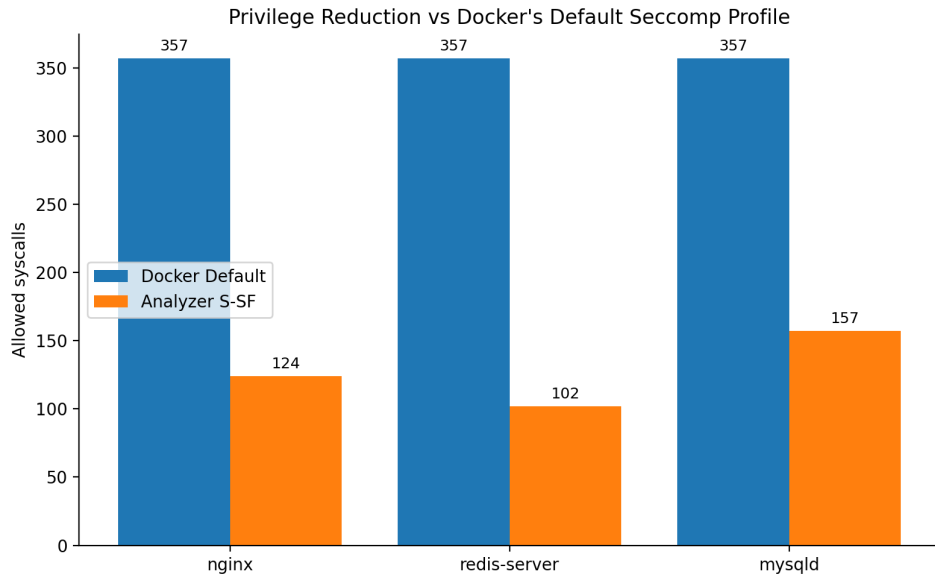


Figure 8: Attack Surface Reduction Relative to Docker's Default Seccomp Profile

The Nginx result (124 syscalls) matches the base paper's reported value exactly, validating the correctness of the core static analysis pipeline. Redis and MySQL results are moderately lower than the base paper's figures (102 vs. 130, and 157 vs. 184, respectively); this is attributed to differences in the specific binary versions analyzed and to differences in libc call-graph completeness between environments, both of which are known, explainable sources of variance in call-graph-based static analysis rather than indications of an implementation defect. Relative to Docker's default profile, the implemented pipeline achieves a 65.3% reduction for Nginx, 71.4% for Redis, and 56.0% for MySQL.

## 6 Metrics for Evaluation

### 6.1 Metrics — Phase I

Table 4: Metrics for Evaluation — Phase I

| Metric                                  | Description                                                                                                                                                                                                                                        |
|-----------------------------------------|----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| Attack Surface Reduction Rate           | Percentage of default Linux syscalls removed by the generated profile: $(1 - \text{Allowed}/\text{Total}) \times 100$ . Measured for the current implementation at 56.0%–71.4% across analyzed binaries, relative to Docker’s 357-syscall default. |
| False Positive Rate (Over-Restriction)  | Percentage of syscalls the application legitimately needs that the profile incorrectly blocks.                                                                                                                                                     |
| False Negative Rate (Under-Restriction) | Percentage of unnecessary or dangerous syscalls the profile mistakenly allows.                                                                                                                                                                     |
| Segmentation Stability                  | Variance in the detected segmentation point across repeated runs and across workload types, compared between fixed-threshold and adaptive change-point detection.                                                                                  |
| Coverage Ratio                          | Ratio of syscalls captured by the combined static and dynamic analysis ( $S\text{-SF} \cup D\text{-SF}$ ) to the true set of syscalls required across all application code paths.                                                                  |
| Static-vs-Baseline Agreement            | Degree of agreement between the implemented static analyzer’s S-SF and the corresponding value reported by the PSF base paper for the same application, used as a correctness sanity check (e.g., exact match of 124/124 syscalls for Nginx).      |

## 6.2 Metrics — Phase II

Table 5: Metrics for Evaluation — Phase II

| Metric                  | Description                                                                                                                                         |
|-------------------------|-----------------------------------------------------------------------------------------------------------------------------------------------------|
| Precision / Recall      | Proportion of flagged anomalies that are true attacks (Precision), and proportion of actual attacks correctly flagged (Recall).                     |
| F1-Score                | Harmonic mean of precision and recall.                                                                                                              |
| False Positive Rate     | Percentage of normal syscall sequences incorrectly flagged as anomalous.                                                                            |
| Argument-Awareness Gain | Improvement in precision, recall, and false positive rate when argument-level features are included, relative to syscall-identifier-only sequences. |
| Detection Latency       | Time elapsed between an anomalous syscall sequence occurring and the risk-scoring engine issuing a response decision.                               |
| Runtime Overhead        | Additional CPU and memory cost introduced by continuous eBPF-based syscall capture and inference, relative to an unmonitored baseline container.    |

## 6.3 Datasets for Phase II Evaluation

Table 6: Datasets Used

| Name                                           | Description                                                                                                                                                                                                                                                   | Size                   | Purpose                                                                                       |
|------------------------------------------------|---------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|------------------------|-----------------------------------------------------------------------------------------------|
| DongTing (DT) Dataset                          | 18,966 labelled Linux kernel syscall sequences — 6,850 normal sequences drawn from four kernel regression test suites, and 12,116 abnormal sequences drawn from bug-triggering kernel fuzzing proofs-of-concept, spanning 26 kernel releases over five years. | 85 GB                  | Candidate training data for the hybrid VAE + iForest anomaly model.                           |
| BETH (BPF-Extended Tracking Honey-pot) Dataset | Real-world, eBPF-collected process creation, clone, and termination events from 23 live honeypots, labelled benign or malicious (e.g., botnet setup, cryptomining, lateral movement), and including per-syscall argument fields.                              | ~928 MB (uncompressed) | Source of argument-level context for the argument-aware feature extraction stage of Phase II. |

## 7 Deliverables

### 7.1 Phase I Deliverables

Table 7: Phase I Deliverables

| Item     | Description                                                                                                                                                            |
|----------|------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| Input    | Container image (e.g., Nginx, Redis, MySQL); extracted binary and shared library files; live syscall traces captured via eBPF.                                         |
| Output 1 | Static Seccomp File (S-SF) — syscalls reachable via binary and call-graph analysis.                                                                                    |
| Output 2 | Dynamic Seccomp File (D-SF) — syscalls observed during live execution.                                                                                                 |
| Output 3 | Initialize Seccomp File (I-SF) — merged S-SF and D-SF, applied at container start.                                                                                     |
| Output 4 | Running Seccomp File (R-SF) — tightened, steady-state syscall profile, generated via adaptive change-point segmentation over frequency and bigram transition features. |

### 7.2 Phase II Deliverables

Table 8: Phase II Deliverables

| Item              | Description                                                                                                                                                               |
|-------------------|---------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| Input 1           | Live syscall stream, captured via eBPF from running containers, already restricted by the Phase I Running Seccomp File.                                                   |
| Input 2           | Labelled syscall sequence dataset (DongTing and/or BETH, supplemented with simulated data as needed) for offline model training.                                          |
| Output 1          | Trained hybrid anomaly model (VAE + Isolation Forest ensemble), trained offline on argument-aware syscall sequence features.                                              |
| Output 2          | Anomaly classification for each evaluated syscall window (Normal Sequence / Anomalous Sequence), together with a risk score (Medium / High) from the risk-scoring engine. |
| Output 3          | Response action — logging and alerting for medium-risk anomalies, active blocking for high-risk anomalies.                                                                |
| Final Deliverable | A hardened, self-defending container: a minimised, adaptively segmented syscall attack surface, combined with continuous, argument-aware behavioural protection.          |

## 8 Project Timeline

The proposed work is planned to be executed across the Academic Year 2026–2027, with Phase I (seccomp profiling) completed in the first half of the year and Phase II (runtime anomaly detection) carried out in the second half, followed by integration, evaluation, and final reporting. The planned schedule is illustrated in Figure 9.

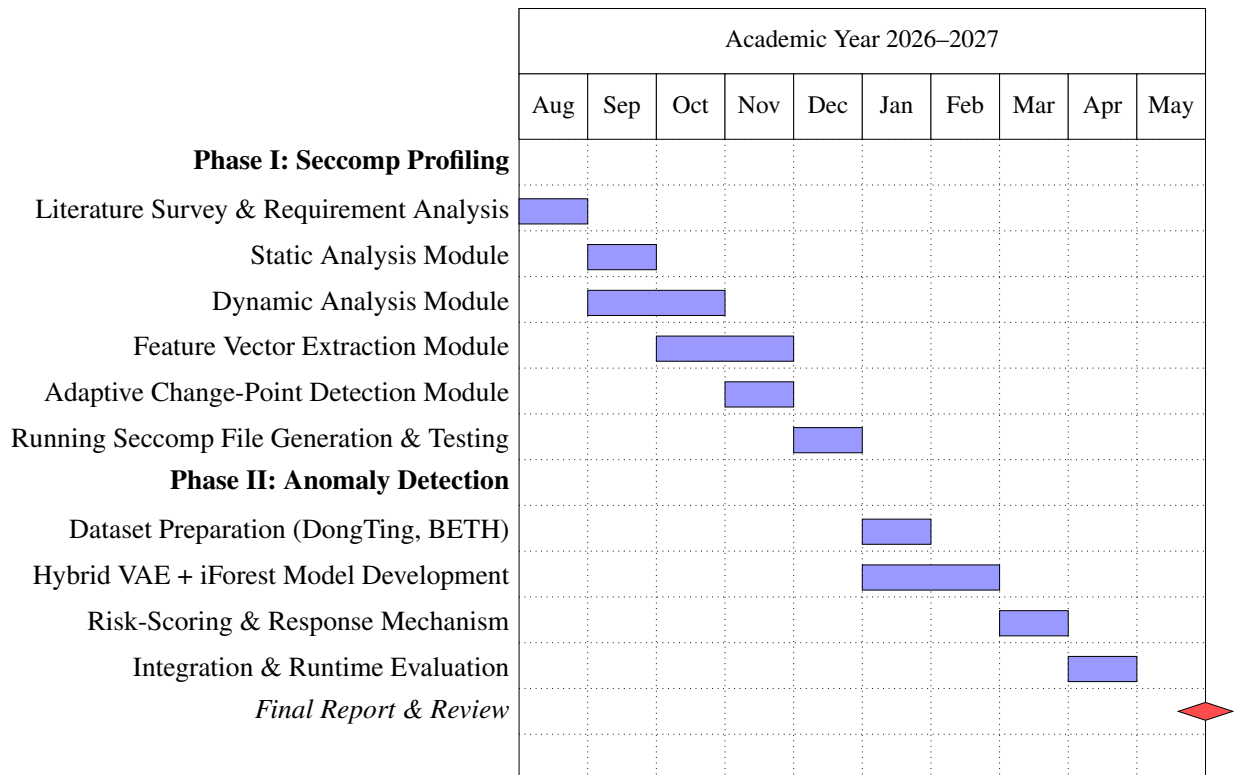


Figure 9: Proposed Project Timeline (Gantt Chart)

## 9 References

1. K. Chen, H. Lu, Y. Yao, B. Fang, Y. Liu, and Z. Tian, “Enhancing Container Security Through Phase-Based System Call Filtering,” *IEEE Transactions on Cloud Computing*, vol. 13, no. 3, pp. 983–994, Jul.–Sep. 2025.
2. H. Kang, E. Marin, M. You, D. Perino, S. Shin, and J. Kim, “BeaCon: Automatic Container Policy Generation Using Environment-Aware Dynamic Analysis,” *Computers & Security*, vol. 162, art. no. 104789, 2026.
3. S. Zehra, H. J. Syed, F. Samad, and U. Faseeha, “DeSFAM: An Adaptive eBPF and AI-Driven Framework for Securing Cloud Containers in Real Time,” *IEEE Access*, vol. 13, pp. 139203–139224, 2025.
4. R. Kim, J. Ryu, S. Kim, S. Lee, and S. Kim, “Detecting Cryptojacking Containers Using eBPF-Based Security Runtime and Machine Learning,” *Electronics*, vol. 14, no. 6, p. 1208, 2025.
5. N. Zhou, H. Chen, Z. Chen, C. Li, and F. Li, “An Abnormal File Access Detection Model for Containers Based on eBPF Listening,” *Mathematics*, vol. 14, no. 6, p. 991, 2026.
6. J. Ryu, R. Kim, S. Lee, S. Kim, H. Choi, and S. Kim, “Hybrid Runtime Detection of Malicious Containers Using eBPF,” *Computers, Materials & Continua*, vol. 86, no. 3, pp. 1–13, 2026.
7. S. Zehra, H. J. Syed, and U. Faseeha, “FedMon: Federated eBPF Monitoring for Distributed Anomaly Detection in Multi-Cluster Cloud Environments,” in *Proc. 2025 20th Int. Conf. Emerging Technologies (ICET)*, Peshawar, Pakistan, 2025, pp. 1–7.
8. H. Taguchi, T. Hara, and S. Kasahara, “Unsupervised Real-Time In-Kernel Intrusion Detection System Using Autoencoders and eBPF,” *IEICE Transactions on Communications*, vol. E109-B, no. 2, pp. 326–336, Feb. 2026.
9. J. Her, J. Kim, J. Kim, and S. Lee, “An In-Depth Analysis of eBPF-Based System Security Tools in Cloud-Native Environments,” *IEEE Access*, vol. 13, pp. 155588–155604, 2025.
10. X. Lin, Z. Chen, W. Yang, X. Yang, J. Luo, and X. Yang, “eBPF-Guard: A Detection Method for Container Escape via Multi-Level Monitoring and Enhanced Analysis Model,” *Empirical Software Engineering*, vol. 31, art. no. 51, 2026.

## Enhancing Container Security Through Phase-Based System Call Filtering

Ke Chen<sup>1</sup>, Hui Lu<sup>2</sup>, Yinnan Yao, Binxing Fang, Yuan Liu<sup>3</sup>, *Senior Member, IEEE*, and Zhihong Tian<sup>4</sup>

**Abstract**—Container technology in cloud computing has improved resource utilization and deployment efficiency, but it also introduces new security risks. Excessive privileges in containerized environments can allow attackers to exploit insufficiently restricted system calls, potentially leading to container escapes and other attacks. Some system calls are only necessary during the initialization phase of a container, and allowing them during runtime can increase the risk of exploitation. This paper proposes a Phase-based System Call Filtering (PSF) method to minimize system call permissions during the runtime of cloud containers. The PSF method builds a comprehensive whitelist of system calls during the initialization phase to cover all necessary calls for containerized applications. During runtime, a refined, phase-specific system call whitelist is enforced, dynamically adjusting privileges based on the functions encapsulated within the container. Additionally, we introduce a container phase recognition algorithm to distinguish between initialization and runtime phases, supporting the generation of phase-specific system call lists. Experimental results show that the proposed method enhances runtime system call restrictions, minimizes privileges, and improves the overall security of cloud containers.

**Index Terms**—Containers, system call filtering, static analysis, dynamic analysis, phased.

### I. INTRODUCTION

CLOUD computing has emerged as a critical solution in modern computing environments. With the rapid growth of cloud services and the increasing demands for data processing, traditional computing resources are increasingly inadequate

to meet the required flexibility and efficiency. As lightweight, portable, and manageable computing units, cloud environments offer enhanced resource utilization and support for distributed, dynamic computational needs, improving the scalability and flexibility of systems. By packaging applications and services into independent modules, cloud environments facilitate seamless operation across heterogeneous hardware and diverse environments. Despite these advantages, cloud environments face significant security challenges, particularly due to the shared use of the underlying operating system kernel, making them more susceptible to attacks compared to traditional virtual machines. [1].

The quintessential delineation between containers and virtual machines hinges on their conjoined host operating system kernel. This inherent shared architecture capacitates applications encapsulated within containers with privileged access to host-centric system resources, encompassing filesystems, networking paradigms, and process management. Notwithstanding the isolation and resource delineation afforded by the Linux kernel through namespace [2] and cgroup [3] frameworks, these foundational isolation paradigms are not impervious and are susceptible to certain security vulnerabilities, encompassing inter-process manipulations, privilege escalation stratagems, and inadvertent disclosure of sensitive information.

To enhance the security of containerized ecosystems, modern container platforms and orchestration frameworks have implemented a range of protective measures, including namespace isolation, filesystem partitioning, and network segmentation. Additionally, to bolster container resilience, Linux-based Mandatory Access Control (MAC) frameworks such as SELinux [4] and AppArmor [5], along with syscall filtering tools like Seccomp BPF [6], have gained widespread adoption. However, despite the robust security mechanisms provided by their default configurations [7], leading container frameworks such as Docker [8] and Kubernetes [9] remain vulnerable to latent security risks due to the complexity inherent in their configurations [10], [11], [12], [13], [14].

In this milieu, the crux of this research endeavors to harness an algorithmic Seccomp syscall filtration stratagem generation framework, aiming to curtail the prospective attack vectors of containers and mitigate the intricacies concomitant with manual policy articulation, thereby championing the ethos of minimal privilege deployment paradigms for containers. The salient contribution of this research is anchored in algorithmic policy synthesis, markedly attenuating the manual labor and inadvertent discrepancies associated with policy articulation, thereby

Received 27 December 2024; revised 18 June 2025; accepted 22 June 2025. Date of publication 26 June 2025; date of current version 8 September 2025. This work was supported in part by the National Key Research and Development Plan under Grant 2023YFB3107300, in part by the National Natural Science Foundation of China under Grant 62272121, Grant U2436208, and Grant 62372129, in part by Guangdong S&T Program under Grant 2024B0101010002, in part by the Project of Guangdong Key Laboratory of Industrial Control System Security under Grant 2024B1212020010, and in part by the Major Key Project of PCL under Grant PCL2024A05. Recommended for acceptance by T. Xiang. (Corresponding authors: Hui Lu; Yuan Liu.)

Ke Chen is with the School of Cyberspace Security, Beijing University of Posts and Telecommunications, Beijing 100876, China, also with the Cyberspace Institute of Advanced Technology, Guangzhou University, Guangzhou 511370, China, and also with the Guangdong Key Laboratory of Industrial Control System Security, Huangpu Research School of Guangzhou University, Guangzhou 510530, China.

Hui Lu, Yinnan Yao, Binxing Fang, Yuan Liu, and Zhihong Tian are with the Cyberspace Institute of Advanced Technology, Guangzhou University, Guangzhou 510530, China, and also with the Guangdong Key Laboratory of Industrial Control System Security, Huangpu Research School of Guangzhou University, Guangzhou 510530, China (e-mail: luhui@gzhu.edu.cn; yaoyinnan@gzhu.edu.cn; fangbx@cae.cn; yuanliu@gzhu.edu.cn; tianzhihong@gzhu.edu.cn).

Digital Object Identifier 10.1109/TCC.2025.3583414

2168-7161 © 2025 IEEE. All rights reserved, including rights for text and data mining, and training of artificial intelligence and similar technologies. Personal use is permitted, but republication/redistribution requires IEEE permission. See <https://www.ieee.org/publications/rights/index.html> for more information.

Authorized licensed use limited to: ANNA UNIVERSITY. Downloaded on July 07, 2026 at 16:43:10 UTC from IEEE Xplore. Restrictions apply.